RED5 AHU

New Controller Setup — Commissioning Runbook

Stack **V1.9** (embedded Delta controller) · Flask port **5001** · Bootloader **/root/data/pgpy/app_main.py**

This runbook takes a factory controller to a running RED5 AHU stack. Steps 1–10 stand up the Flask bootloader, steps 11–13 deploy the rest from `red5_bundle.zip`, and steps 14–16 bring up the telemetry collector.

You never paste a large script into enteliWEB. Both PG objects hold a single line that loads the real code from disk. Pasting the full bootloader fails with `QERR_CODE_NO_SPACE` on current hardware — see step 8.

The Debug Console executes one line at a time. Every diagnostic in this document is written as a single physical line for that reason. If your editor wraps one across several lines when you copy it, re-copy it — a multi-line paste will be rejected.

PART A — BOOTLOADER (ENTELIWEB)

1 **Get a new controller**

Rack and power the controller, and put it on the same network segment enteliWEB can reach. Note its IP address — you will need it from step 10 onward.

2 **Discover the controller in enteliWEB**

In enteliWEB, run a device discovery and confirm the new controller appears with the expected device instance and name before continuing.

3 **Set the controller time**

Correct the controller's date, time, and time zone.

> **Why this comes early:** telemetry snapshots, the band CSV refresh, and the weather history cache are all keyed by timestamp. Commissioning on a wrong clock produces data that looks corrupt later and is tedious to unwind.

> **Check the time zone, not just the time.** A controller commissioned in Sep 2026 was found logging in UTC — an 8-hour offset from site local time. The stack runs fine that way, so nothing alerts you; the skew only shows up later in trend history.

4 Open the DEV object and enable PG Python

Open the controller's **DEV** (device) object and enable **PG Python**. Python programs cannot be created or run until this is on.

5 Create the PG object named “app” in Python mode

Create a new **PG** (Program) object, name it **app**, and set its program mode to **Python** using the **Mode** button in the editor toolbar. This becomes `/root/scripts/app.py`.

> **Program mode is the single most common setup mistake.** If the mode is not Python, the object still accepts and saves your text, but every run fails with `Programming Error` — even a bare `print()` — and no file is ever created in `/root/scripts/`. See *Troubleshooting*.

Verify the object is real before going further:

```
import os; print(sorted(os.listdir('/root/scripts')))
```

You want `['app.py']`. If the list is empty, the mode is wrong — fix it now rather than debugging code later.

> **The name matters.** The firmware deletes any `.py` in `/root/scripts/` that is not a registered enteliWEB object. Registering it as `app` is what makes the bootloader persist.

6 Install the Python dependencies

Install the two required modules — and only these two:

```
pip install flask
pip install flask_cors
```

> **Do not install Pillow.** It only adds thumbnail normalisation at `/api/thumb`, which the bootloader imports lazily and gracefully skips — falling back to redirecting to `/assets/`. Pillow and its dependencies can consume tens of megabytes of flash and buy you nothing on an embedded controller.

Check free space:

```
import shutil; u=shutil.disk_usage('/root'); print('FREE %.2f MB' %
(u.free/1048576))
```

7 Wait for the installs to finish

Let each `pip install` run to completion before moving on. Starting the bootloader while a module is still installing gives a misleading `ImportError` on first run.

8 Stage the bootloader on disk

The bootloader is ~90 KB and **must not be pasted into the PG object**. Put it on disk instead, where there is no size limit.

Source A — an already-commissioned controller (preferred; no laptop needed). Run this in the Debug Console of the *new* controller:

```
import urllib.request, os; os.makedirs('/root/data/pgpy', exist_ok=True);
urllib.request.urlretrieve('http://<working-controller-
ip>:5001/assets/app_canonical_c2.py.txt', '/root/data/pgpy/app_main.py');
print('bytes:', os.path.getsize('/root/data/pgpy/app_main.py'))
```

Source B — your Mac. Serve the folder first:

```
cd /Users/jinkim/red5/red5-ahu/archive/Red5-AHU-V1.9
python3 -m http.server 8000
```

Then, in the Debug Console:

```
import urllib.request, os; os.makedirs('/root/data/pgpy', exist_ok=True);
urllib.request.urlretrieve('http://<mac-ip>:8000/app_canonical_c2.py',
'/root/data/pgpy/app_main.py'); print('bytes:',
os.path.getsize('/root/data/pgpy/app_main.py'))
```

Expect `bytes: 89565`. A smaller number means a truncated download — delete the file and retry rather than continuing.

> **Why not paste it?** On a Red5-PLUS-1146 the full paste fails with `Execution_Code: QERR_CLASS_OS::QERR_CODE_NO_SPACE` while the filesystem still has tens of megabytes free. The limit is on *program text stored in the object*, not on flash. The object silently keeps only the first part of the script, which looks like a successful save until it fails at runtime.

9 Paste the one-line loader into “app”, then Run

The entire contents of the **app** PG object is this single line:

```
exec(compile(open('/root/data/pgpy/app_main.py').read(),
'/root/data/pgpy/app_main.py', 'exec'))
```

Save — it commits instantly, with no “loading” spinner — then set the object to **Run**.

> **This object never needs editing again.** Future bootloader upgrades replace `/root/data/pgpy/app_main.py` on disk; the loader line stays as is. The bundle ships that file, so after step 11 it is kept current automatically.

> **enteliWEB does not auto-respawn this object.** If the Python process ever stops, it stays stopped until an operator Starts it again. Remember this for step 12.

10 Confirm the output window

On a factory controller no plug-ins exist yet, so the bootloader installs its emergency bootstrap and tells you where to go next. Expect output of this shape:

```
[service-discovery] no *_service.py files found in ['/root/data/pgpy',
'/root/scripts']
[bootstrap-fallback] No upload service registered - installing emergency
bootstrap so the operator can deploy upload_service.py via /update

* Controller Online at: http://<controller-ip>:5001
* Update page at: http://<controller-ip>:5001/update
```

Those last two lines confirm a healthy boot. The `no *_service.py files found` line is expected here — it disappears once the bundle is deployed.

PART B — DEPLOY THE STACK (BROWSER)

11 Upload the bundle at `/update`

Browse to `http://<controller-ip>:5001/`. Because `landing.html` does not exist yet, you are redirected to `/update` and served the red **Emergency Bootstrap** page. Upload `red5_bundle.zip` there.

> **The password field accepts anything non-empty.** `red5_bundle.zip` is a plain zip — it begins with PK, not RED5ENC1/RED5ENC2 — so the bootstrap skips decryption entirely and never checks the value. The form only refuses an empty field. A real key is needed solely for encrypted bundles produced by the Download/Replicate feature.

The extractor routes `.py` plug-ins to `/root/data/pgpy/` and everything else to `/root/data/`. It deliberately refuses any bundled `app.py`, so your bootloader cannot be clobbered.

> **This upload also installs the master key** to `/root/data/master_key.txt`, and it activates `auth_service.py`. If a later upload is rejected with `Authentication required`, see *Troubleshooting*.

12 Stop and Start the “app” object

Plug-ins are imported once at boot, so nothing you just uploaded is live yet. Stop the **app** PG object in enteliWEB and Start it again.

The output window should now list one `registered OK` line per plug-in, and the `bootstrap-fallback` message should be gone — the full `/update` UI is now served by `upload_service.py`.

13 Verify

Check these in order. All paths are relative to `http://<controller-ip>:5001`.

ENDPOINT	EXPECTED RESULT
`/api/services`	Every plug-in reports `OK`. Any `FAILED` entry includes a full traceback in the JSON.
`/api/version`	Non-null mtimes for `upload_service.py` and `pages_service.py`.
`/api/health`	`{"ok": true, "version": "1.9.0"}` — proves the bundled plug-in routes loaded.
`/`	The access page, no longer a redirect to `/update`.
`/dashboard`	Dashboard renders.
`/mobile`	Phone-first view renders.

> **Expected non-failure:** `/api/download-bundle/app.py` returns 404. The bootloader is never copied to `/root/data/`, so that whitelist entry has nothing to serve.

PART C — TELEMETRY COLLECTOR

`collector.py` polls BACnet points via `dibt.Read()` and writes `telemetry.json`, which the dashboard reads. It must be its own PG object: `dibt` is injected only into registered enteliWEB objects, so a plug-in thread inside `app` cannot reach it. The bundle already placed the code at `/root/data/pgpy/collector.py` in step 11 — you are only creating the object that runs it.

14 Create the PG object named “collector” in Python mode

Exactly as in step 5: a new **PG** object named **collector**, program mode set to **Python** via the **Mode** button. Then verify:

```
import os; print(sorted(os.listdir('/root/scripts')))
```

You want `['app.py', 'collector.py']`. If `collector.py` is absent, the program mode is not Python — nothing you paste will ever run.

> **Do not create that file yourself.** Writing to `/root/scripts/` from Python appears to work and is then silently deleted by the firmware. Only the registered-object workflow puts a file there.

15 Paste the one-line loader into “collector”, then Run

```
import sys; sys.argv=['collector.py'];
exec(compile(open('/root/data/pgpy/collector.py').read(),
'/root/data/pgpy/collector.py', 'exec'))
```

> **The `sys.argv` assignment is required.** `collector.py` builds an `argparse` parser, and argparse reads `sys.argv[0]` for its program name. A PG object has no command line, and an empty `sys.argv` raises `IndexError: list index out of range` before any of the collector's own code runs.

> **Do not append `main()`.** `collector.py` calls it on its own last line. Adding a second call starts the poll loop twice.

Expected `PG Output`:

```
Red5 Telemetry Collector v1.2 starting...
Config path: /root/data/configs/collector_config.json
Mock mode: False
Telemetry output: /root/data/configs/telemetry.json
Discovered 0 AHU groups
Poll interval: 5s
```

16 Verify the collector is polling

Browse to `http://<controller-ip>:5001/api/telemetry-status`. You want `age_seconds` under about 10 and `stale: false`. A large `age_seconds` that keeps growing means the object is not actually running.

> **`Discovered 0 AHU groups` is expected on a fresh controller.** It means `map_config.json` has no equipment wired yet. Complete the equipment mapper before expecting live data — this is configuration, not a deployment fault.

TROUBLESHOOTING — PROGRAMMING ERROR ON EVERY RUN

Symptom: the object saves your text without complaint, but **Run** immediately shows `Program_Change : Programming Error` and **Halted**. The decisive test is to replace the whole program with one harmless line:

```
print('hello')
```

If even that fails, the program is not being run as Python. Confirm with:

```
import os; print(sorted(os.listdir('/root/scripts')))
```

A missing entry for your object proves it: the firmware only materialises `/root/scripts/<name>.py` for Python-mode programs. Fix the **Mode** setting rather than editing the script. This was the root cause of a

multi-hour collector outage in Sep 2026, during which the script, the file path, and the loader line were all repeatedly — and wrongly — suspected.

TROUBLESHOOTING — QERR_CODE_NO_SPACE

Symptom: saving a large pasted script raises `Execution_Code: QERR_CLASS_OS::QERR_CODE_NO_SPACE`. This is a limit on how much **program text a single PG object can store**, not free flash — it reproduces on a Red5-PLUS-1146 with tens of megabytes free. Worse, the object may keep only the first portion of the paste, so the save looks like it worked and fails at runtime instead.
The fix is structural, not a matter of reclaiming space: keep the object at one line and load the real code from disk, as in steps 8, 9 and 15. If you still want to confirm the filesystem is healthy:

```
# free space
import shutil; u=shutil.disk_usage('/root'); print('total %.1f MB / used %.1f MB / FREE
%.2f MB' % (u.total/1048576, u.used/1048576, u.free/1048576))

# 25 largest files under /root
import os; ps=[os.path.join(r,f) for r,d,fs in os.walk('/root') for f in fs];
print('\n'.join('%9.1f KB  %s' % (os.path.getsize(p)/1024, p) for p in sorted(ps,
key=os.path.getsize, reverse=True)[:25]))
```

If space genuinely is short, reclaim it with `pip uninstall -y Pillow` (keep only `flask` and `flask_cors`), then clear `__pycache__` directories and stale `.tmp` or upload-spool files under `/root/data/`.

TROUBLESHOOTING — AUTHENTICATION REQUIRED AT /UPDATE

This is *not* the bundle password — that field is unchecked for plain zips. The message comes from `auth_service.py`, which the bundle installs in step 11, and it appears only when enforcement has been switched on. Check:

```
import os; p='/root/.red5/auth_settings.json'; print(os.path.exists(p) and open(p).read())
```

If that shows `{"enforce": true}`, log in as `admin` — that reserved account authenticates with the master key, which the bundle installed on the controller:

```
print(open('/root/data/master_key.txt').read().strip())
```

To return to the documented default of report-only instead:

```
open('/root/.red5/auth_settings.json','w').write('{"enforce": false}')
```

Hard-refresh afterwards. Re-enable later with `POST /api/auth/enforce {"enable": true}` once real users exist on this controller.

COMMISSIONING THE NEXT CONTROLLER

Once this controller is up it serves the bootloader text itself, so you never need the zip or the repository again — this is Source A in step 8:

```
http://<this-controller-ip>:5001/assets/app_canonical_c2.py.txt
```